

04

SQL Agent with Tool Use

A ReAct-style agent that reasons over database tools instead of generating one query blind

► think → act → observe → repeat

No schema is handed to the model up front. Given only a question, the agent decides for itself whether it needs to list tables, inspect a table's columns, or run a query — calling one narrow tool at a time and reading the result before deciding its next move. A failed query is not a special case; it's just another observation fed back into the same loop.

WORKFLOW

- 1 Think
- The question is classified against a small set of intents (revenue by region, top customers, best-selling products, order status, monthly trend, raw SQL pass-through) to form a thought and a plan before any tool is touched.
- 2 Act
- The agent calls `list_tables()`, then `describe_table(table)` for each table its plan actually references, discovering the schema through tool calls rather than being handed it.
- 3 Observe
- Each tool call is tracked as running → done/error with its raw output and returned rows, rendered as an inspectable card in the chat — never silently swallowed.
- 4 Answer
- Once `run_sql_query` returns, the rows are summarized into a plain-English answer alongside the executed SQL and a result table.

TOOLS EXPOSED TO THE AGENT

<code>list_tables()</code> Lists every table in the database with its row count. Always the first call for a natural-language question.	<code>describe_table({ table })</code> Returns column names and types for one table. Called only for tables the plan actually joins.	<code>run_sql_query({ sql })</code> Executes a read-only SELECT / WITH statement and returns rows, or an error observation.
--	---	--

VS REFERENCE•

Reference (and Experiment 1) generates a full query in one shot against tables chosen by embedding retrieval. This agent doesn't know the schema at all until it asks — schema access is a tool call, not a precondition.

- Failure handling isn't a separate repair function — a blocked or malformed query surfaces as an *error* status on the tool call, the same observation shape the loop already handles.
- The trace is inherently multi-step and inspectable: every tool call is a discrete, expandable card showing its input, status, and returned rows — not one LLM call hidden behind a spinner.

IMPLEMENTATION

```
src/lib/agent.ts

1 export async function runAgent(
2   question: string,
3   onUpdate: (partial: Partial<AgentMessage>) => void,
4 ): Promise<void> {
5   // raw SQL pasted in? skip planning, execute read-only
6   const isRawSql = /^(select|with)\s/i.test(question.trim());
7   const plan = isRawSql ? { thought: "Already SQL — execute directly.", sql: question }
8                         : planFor(question);
9
10  onUpdate({ thought: plan.thought });
11
```

```
12   if (!isRawSql) {
13     push(tool("list_tables", {}));
14     for (const table of tablesReferencedBy(plan.sql)) {
15       push(tool("describe_table", { table }));
16     }
17   }
18
19   const result = executeTool("run_sql_query", { sql: plan.sql });
20   if (result.status === "error") {
21     return onUpdate({ content: `The query failed: ${result.error}` });
22   }
23   onUpdate({ content: plan.answer(result.rows), rows: result.rows, sql: plan.sql });
24 }
```

src/lib/db.ts — run_sql_query guardrail

```
1  const FORBIDDEN = /\b(insert|update|delete|drop|alter|create|truncate|attach)\b/i;
2
3  export function runQuery(sql: string): Record<string, unknown>[] {
4    const trimmed = sql.trim().replace(/;$/, "");
5    if (!/^select\s|^with\s/i.test(trimmed))
6      throw new Error("Only SELECT queries are allowed.");
7    if (FORBIDDEN.test(trimmed))
8      throw new Error("Write statements are blocked in read-only mode.");
9    return alasql(trimmed) as Record<string, unknown>[];
10 }
```

database → in-browser alasql, three tables: *customers* · *products* · *orders*, seeded with 8 customers, 7 products, 15 orders

READ-ONLY GUARDRAILS

STATEMENT WHITELIST

Only strings starting with *SELECT* or *WITH* are accepted — everything else is rejected before it reaches alasql.

KEYWORD BLOCKLIST

INSERT / UPDATE / DELETE / DROP / ALTER / CREATE / TRUNCATE / ATTACH are pattern-matched and blocked even inside a nested clause.

ERROR AS OBSERVATION

A blocked query doesn't throw the loop — it becomes an *error* status on the tool call and a plain-English explanation for the user.

SIDE-BY-SIDE ADDITION

EXPERIMENT	DATA FED TO THE LLM	DISTINGUISHING FEATURE
SQL Agent with Tool Use	Question only, upfront — schema discovered via list_tables / describe_table calls	Multi-step tool-calling loop over a read-only sales DB; blocked/failed SQL is just another observation, rendered as inspectable tool-call cards

BUILT WITH

TanStack Start

React + TypeScript

Tailwind CSS

alasql (in-browser SQL)

shadcn/ui

Lovable